pfi

# Python for Fixed Income

COURSE 10 · IMPLEMENTING RAG

# A Filing as Pages and Items

WEEK 1 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# Today: a filing as pages and items

## One filing, two copies

| | |
|---|---|
| **PDF** | the course's print, 113 pages |
| **HTML** | the 10-K as filed (Course 7) |
| **Pages** | what a reader opens and cites |
| **Items** | Item 1 to Item 16; never crossed |
| **Model** | none today; nothing is sent |

## Today

1. Why a PDF
2. pdf_pages: the pages
3. Four pages read as text
4. Pages against the HTML
5. page_items: the items
6. Printed pages against PDF pages
7. table_like_lines; a debt table

**Look at the pages before anything is chunked: a chunker sees only what they hold.**

# Why load a PDF when the HTML is there

### Two reasons from the desk

Most documents reach a desk as PDFs: offering memoranda, indentures, agency reports.

Pages are what a reader cites: page 37 of a stated file can be opened and checked.

**7,729,841**

bytes in the PDF: fonts and line positions

**4,008,734**

bytes in the HTML as filed

**422,118**

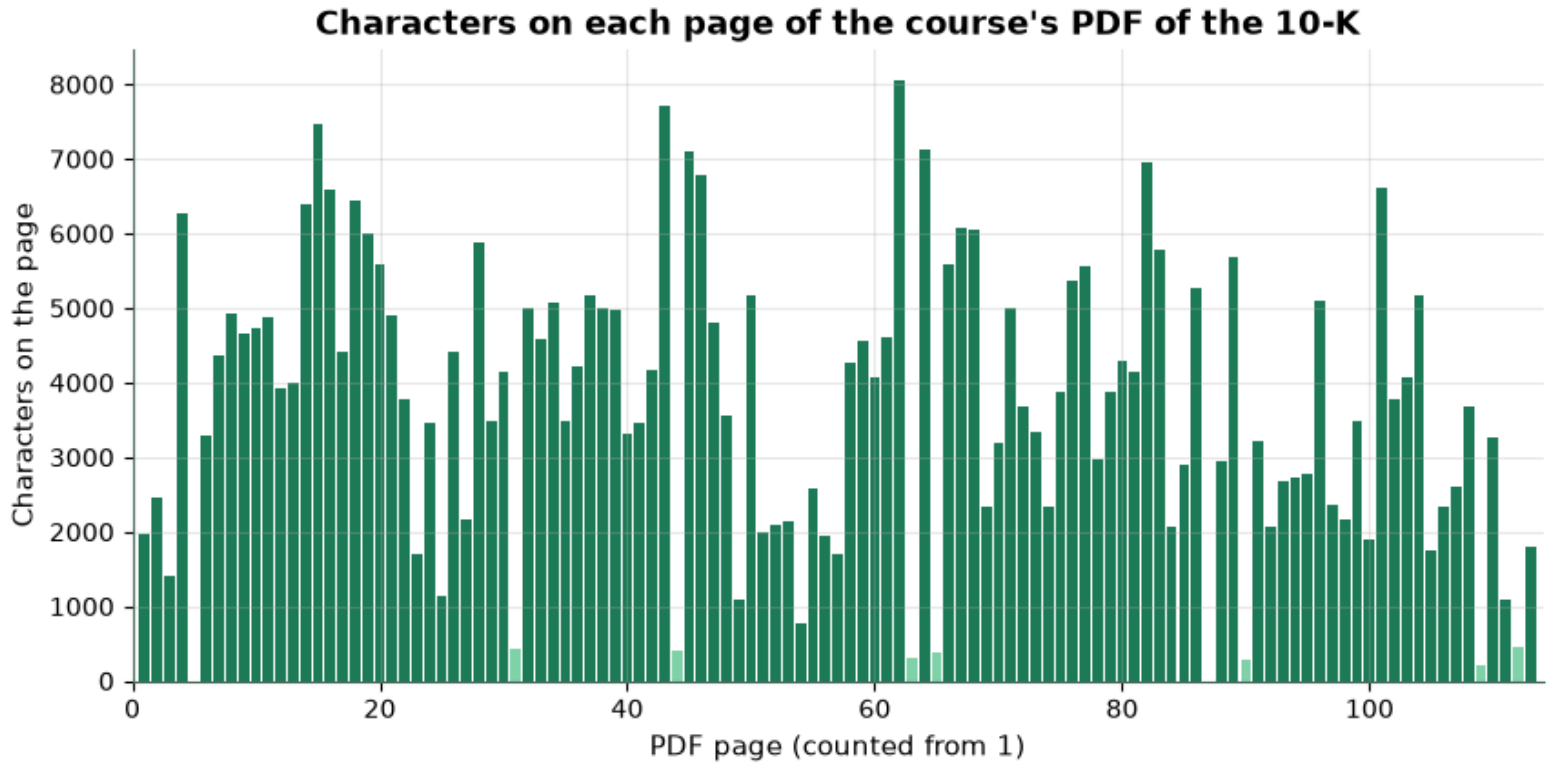
characters of HTML text, tags removed

**EDGAR lists no PDF of this 10-K: the course printed one, and cites its pages.**

# How pdf_pages works

| | |
|---|---|
| **The problem** | A citation needs a page; a PDF is a stack of pages, each an image with a text layer under it |
| **The read** | textkit.pdf_text page by page (pypdf, then normalize_text): the text layer, look-alikes made plain |
| **One row a page** | Page counted from 1, text, characters, the printed number when the last line holds one |
| **Empty pages** | Kept as a row of 0 characters, so every page after keeps its number |
| **What comes out** | A DataFrame of four columns; you control only the path, the numbers are the file's |

# The PDF as pages




## pdf_pages found

**113**
pages

**421,643**
characters

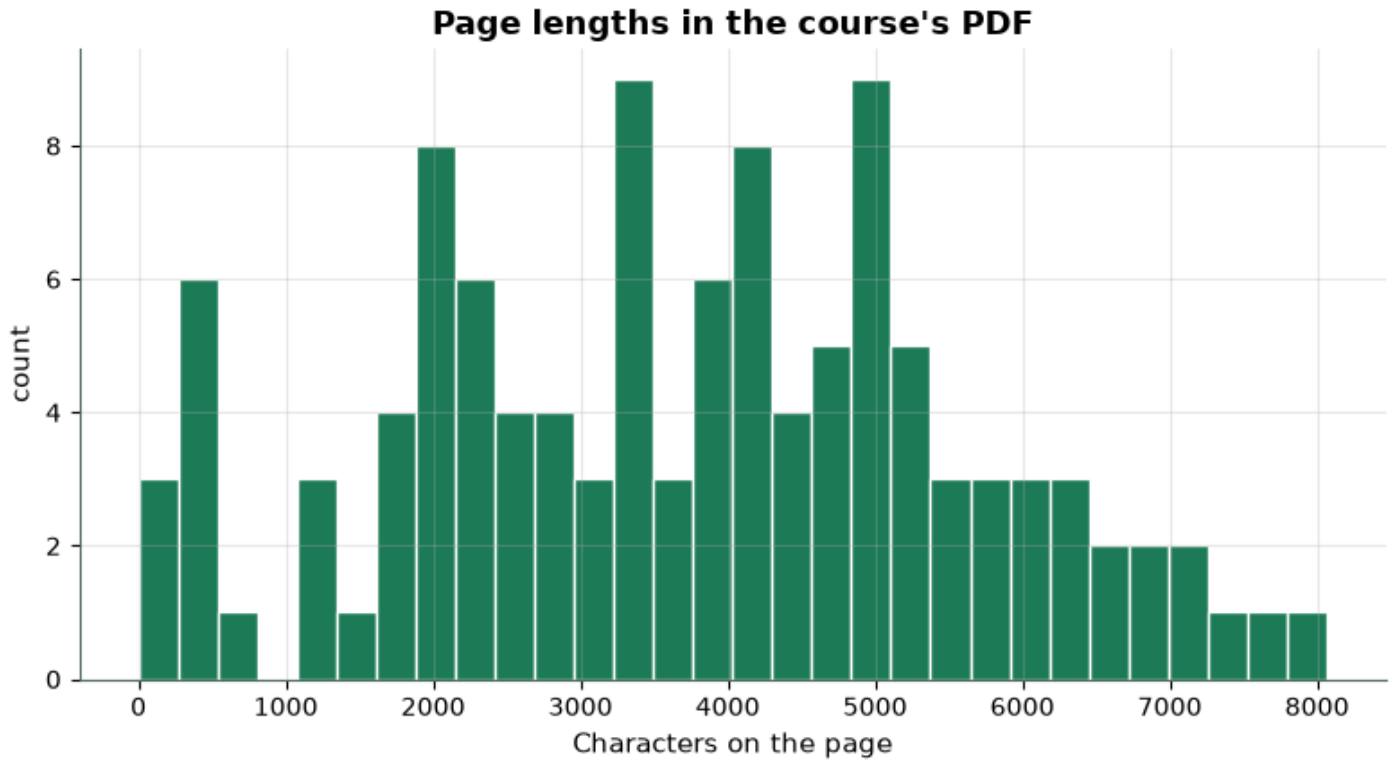
**3,731**
mean per page

**page 87**
empty, kept

**8 pages**
under 500 characters

**102**
end with a printed number

# Page lengths




### A typical page

**3,775 characters**

a median page; day 3's chunks are at most 512 characters each

### The empty page

Page 87 has no text layer. Kept as a row of 0 characters, so page 88 stays 88.

# Reading pages as text

**p. 1  The cover**

form text, box characters, and an image's file name

**p. 3  The contents**

items with the 10-K's printed page numbers

**p. 38  Item 7 prose**

line breaks where the print put them

**p. 74  The debt table**

Note 7: rows intact, column heads broken

## 105 of 113

pages start with the line "Table of Contents"

## 7

short sheets a printed page ran onto

## page 87

empty: no header, no text

**A chunker sees exactly what pdf_pages returns, running header included.**

# Pages against the HTML

```
squeeze = lower case, letters and digits only
found = squeeze(page) in squeeze(html_text)
```

## 111 of 113

pages found in the HTML, letter for letter

## page 1

holds "logoboyd.jpg": the logo's file name

## page 25

holds "byd2025.jpg": a graph's file name

An empty page passes trivially: an empty text is inside any text.

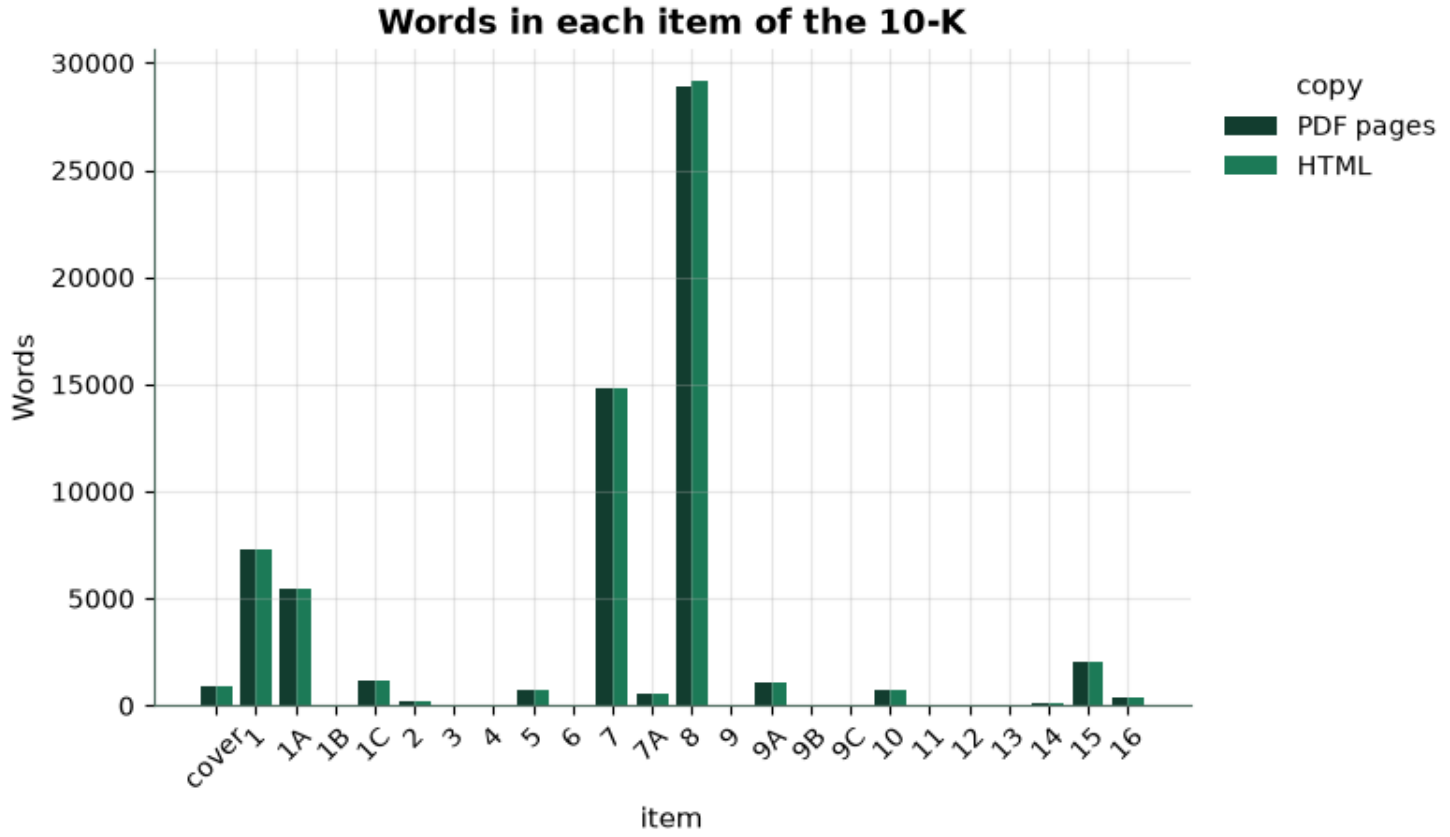
So the check proves the PDF adds nothing; it cannot prove the PDF dropped nothing.

**The PDF's text is the filing's text; the print moved line breaks and page breaks.**

# How page_items works

| | |
|---|---|
| **The problem** | Pages carry no mark of their item, and a chunk must never cross from one item into the next |
| **The headings** | Found as split_items finds them: a line starting with the label, searched from the last item back |
| **The walk** | Pages in order, the current item carried along; a page is cut in two where a heading falls inside it |
| **You control** | TEN_K_ITEMS and the items table from the HTML; the headings are the filing's |
| **What comes out** | One row per page, two or more at a boundary: page, item, text, length; the pieces a chunker cuts inside |

# Items: where each section starts and ends




## page_items

**23**

items found, none missing

**136**

rows from 113 pages

**13**

pages that hold two items or more

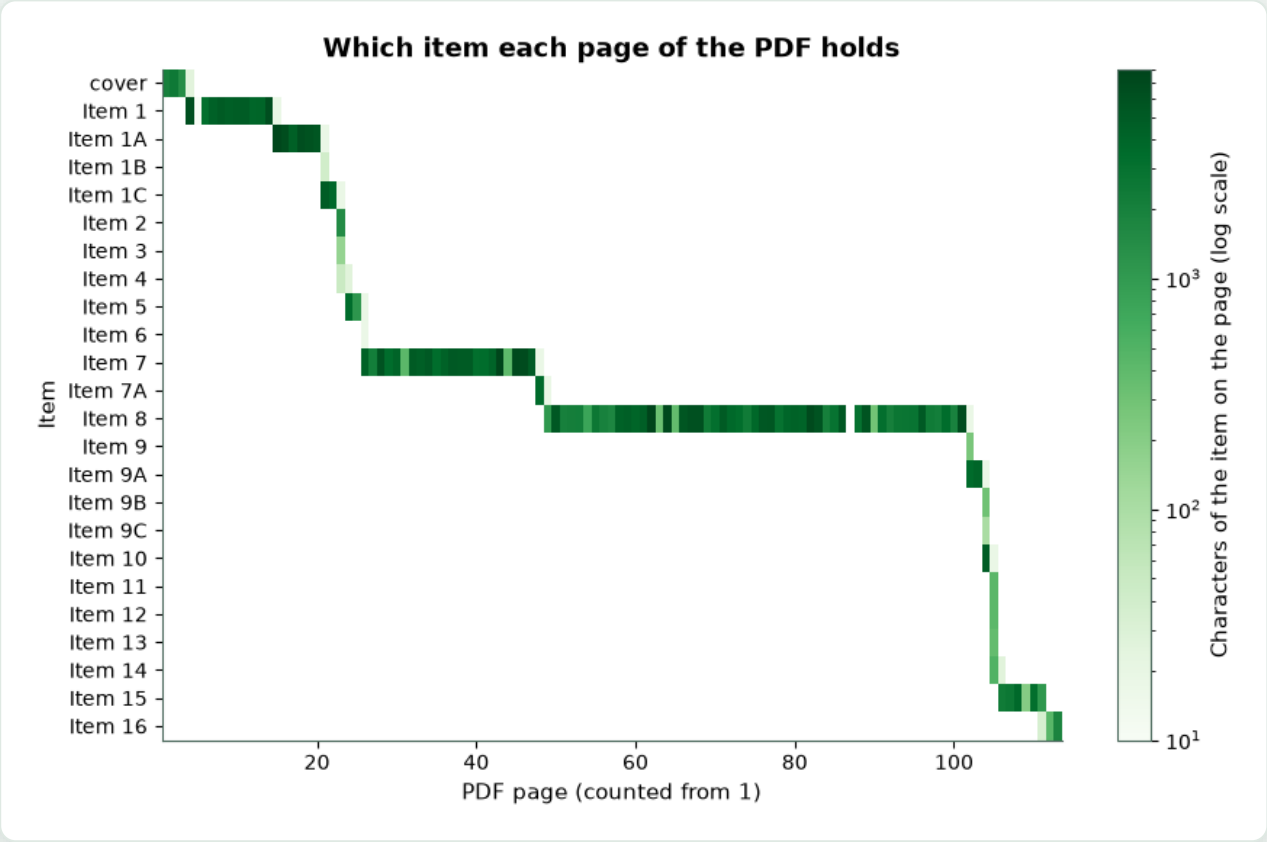
**5 on p. 105**

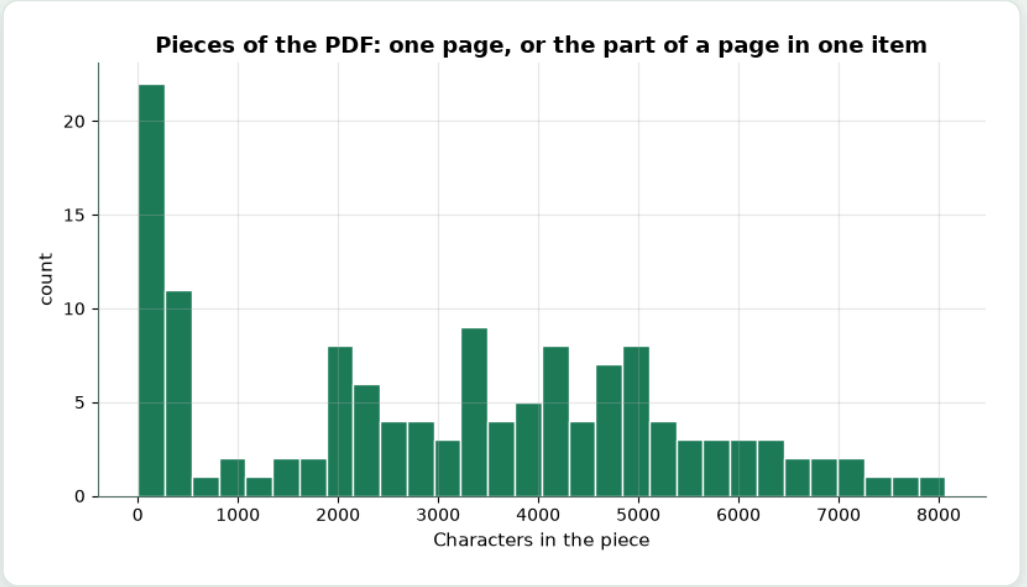
the Part III items

**never**

a chunk across two items

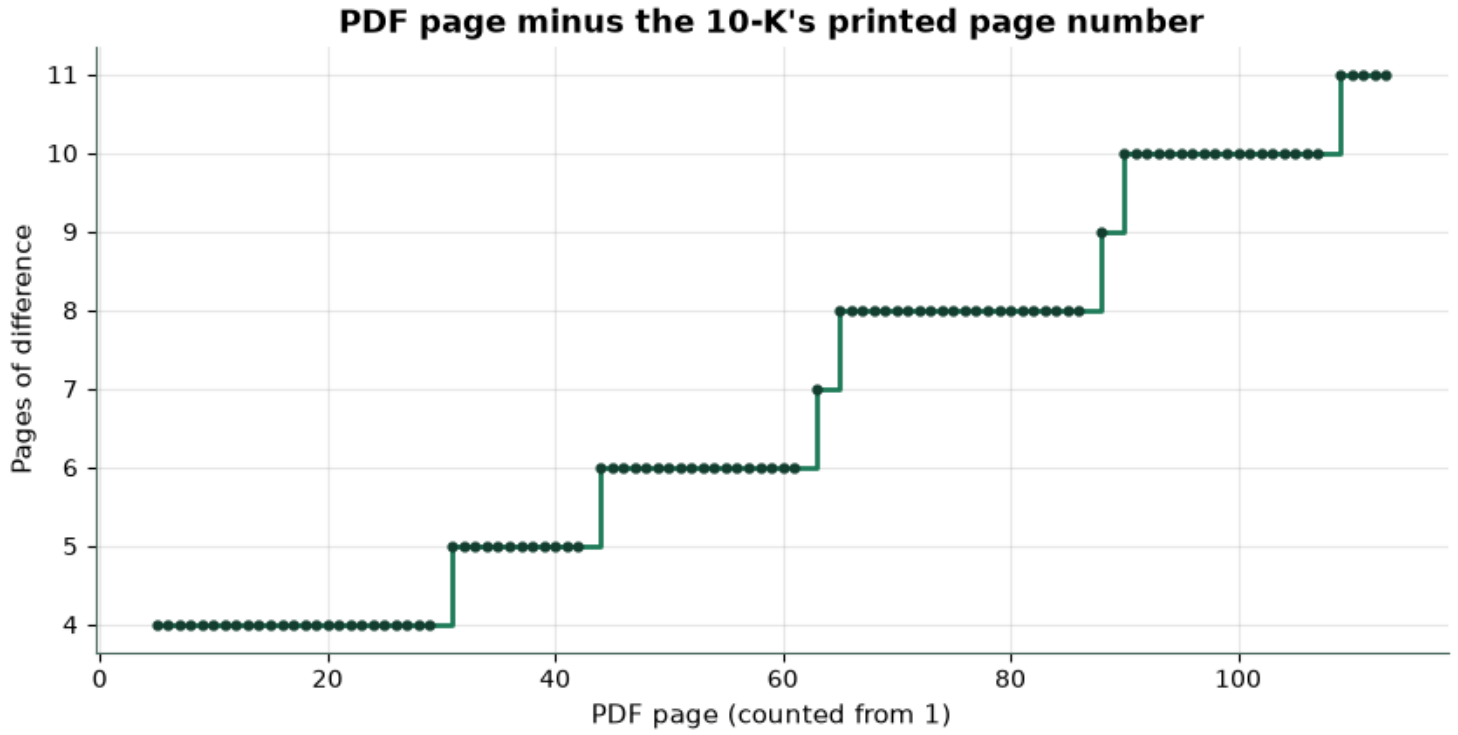
# A page-by-item map, and the pieces a chunker sees




**136 pieces; 18 under 100 characters; 9 only "Table of Contents".**

# Printed page numbers against PDF pages




## What the pages show

**23 of 23**

headings at the contents' page

**+4**

on the first numbered page

**+11**

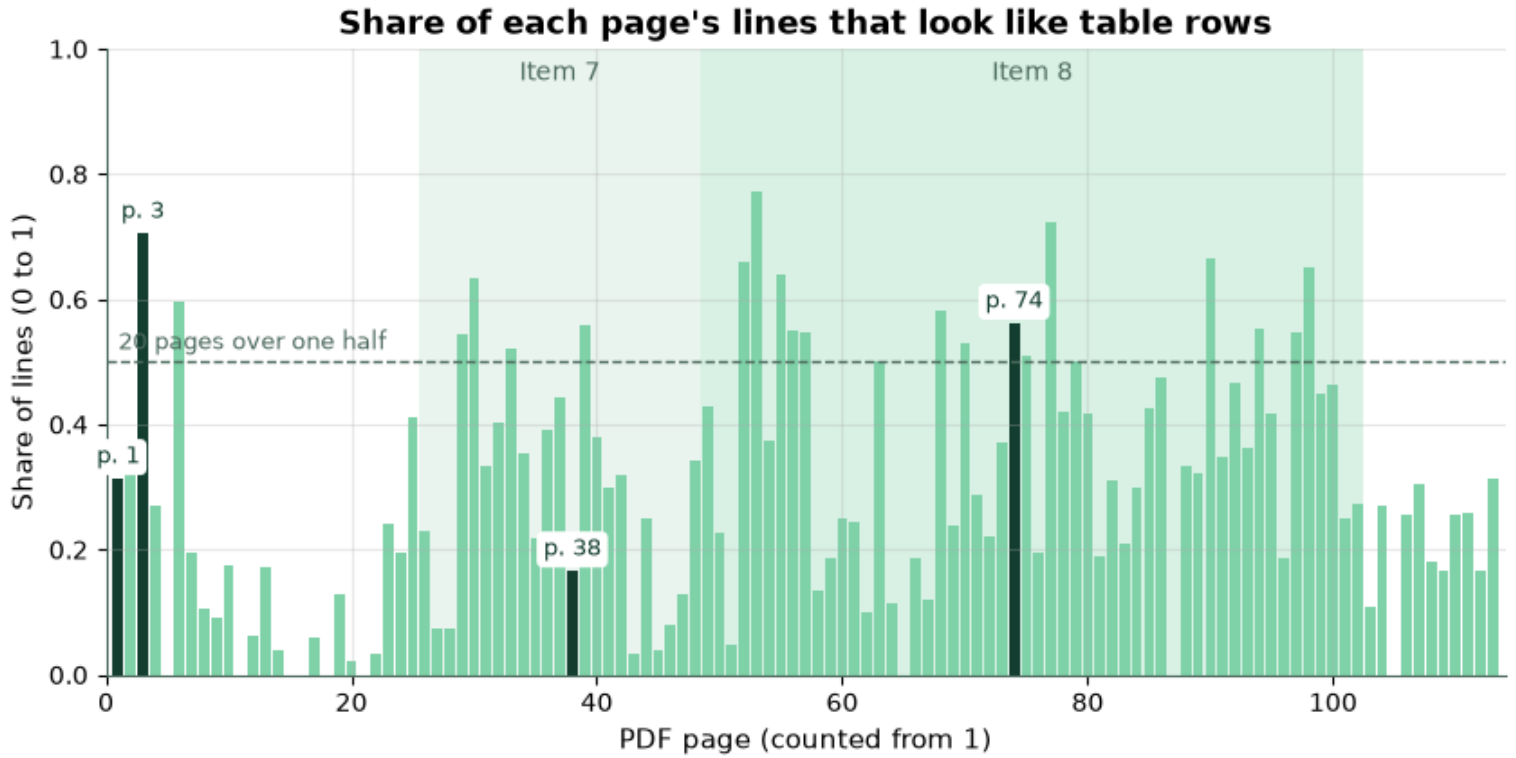
on the last

**1 to 102**

the 10-K's printed pages

**The course cites the PDF's page: the one anyone holding the file can open.**

# Table-like lines on each page




## The four pages read

| | |
|---|---|
| p. 1 cover | **0.3143** |
| p. 3 contents | **0.7059** |
| p. 38 Item 7 | **0.1667** |
| p. 74 debt table | **0.5625** |

Mean 0.2929; 20 pages over one half; 0 lines with a run of three spaces.

# What a text layer does to a debt table

### Page 74, from Note 7 (quoted as published)

```
NOTE 7. LONG-TERM DEBT

Long-term debt, net of current maturities and debt issuance cos...

December 31, 2025

Interest Unamortized

Rates at Origination

December 31, Outstanding Fees and Long-Term

(In thousands) 2025 Principal Costs Debt, Net

Credit facility 5.318% $ 160,700 $ (3,820) $ 156,880

4.750% senior notes due 2027 4.750% 1,000,000 (3,896) 996,104

4.750% senior notes due 2031 4.750% 900,000 (7,415) 892,585

Long-term debt, net $ 2,060,700 $ (15,131) $ 2,045,569
```

### Page 75, its top

```
Table of Contents

BOYD GAMING CORPORATION AND SUBSIDI...

NOTES TO CONSOLIDATED FINANCIAL STA...

as of December 31, 2025 and 2024 an...

December 31, 2024

Interest Unamortized

Rates at Origination

December 31, Outstanding Fees and L...
```

**The 2024 block starts here; "NOTE 7" is on page 74 only.**

Source: Boyd Gaming Corporation, Form 10-K for fiscal 2025, filed 2026-02-20, accession 0001437749-26-004908, Item 8, page 74 of the course's PDF.

# Excel side by side, and Git

**Excel side by side (tested in Excel)**

| | |
|---|---|
| `=LEN(A2), filled down (Text cells)` | equal on 48 of 48 lines |
| `=SUM(B2:B49)` | 2,299, equal to Python |
| `=COUNTA(A2:A49)` | 48 lines, equal |
| `"December 31, 2025" in a General cell` | a date: 46022; LEN 5, not 17 |

**Format the column as Text before you paste a page. An item over 32,767 characters needs several cells.**

**Git: one commit of nine files: the modules, ragkit with 14 tests, .gitignore, and pages.csv.**

# The AI check

The docstring is the prompt. The retrieval code that comes back runs and looks finished.

1

## Docstring

Units, conventions, and what the code must never let through: what you asked for.

2

## Code back

Written for the course in the style of an AI answer. Read it.

3

## Test first

Plant the case the docstring promises to catch. Run it: it fails.

4

## Fix

Find the rule it broke, fix it, run the test again.

**Today: an empty page is skipped, so every later page is numbered one early.**

*Code that runs is not the same as code that is right.*

# What you can do now

- Say why a desk loads a PDF, and cites its pages
- Read every page with pdf_pages, the empty one kept
- Check a PDF against the HTML; say what that proves
- Place pages in items with page_items; cite the PDF's page
- Measure table rows; say what a text layer loses

**Week 1, Day 3** Chunking: size and overlap